دانشگاه صنعتی خواجه نصیرالدین طوسی

Traffic Controller
Project Report

Fatemeh Amirabadi

Introduction

This report presents the design and implementation of a four-way traffic light controller for a road intersection using the **ATmega64** microcontroller and **AVR Assembly**. The intersection includes two perpendicular directions: **North–South (NS)** and **East–West (EW)**, each with **Red**, **Yellow**, and **Green** signals. The controller runs continuously, supports real-time timing adjustment via external interrupts, and includes a traffic sensor mode to prioritize the congested direction.

Objectives

- Implement an intersection controller based on a **Finite State Machine (FSM)** with deterministic timing.
- Use **Timer1 in CTC mode** to generate a precise periodic tick (10 ms).
- Use **INT0, INT1, INT2** to adjust timing parameters with short/long press detection.
- Enforce minimum/maximum timing constraints to prevent invalid values.
- Implement a **traffic sensor mode** using PC1..PC0 and restore original timings after traffic clears.
- Validate behavior through **Proteus simulation**.

System Overview and Pin Mapping

0.0.1 I/O Assignment

The following pin mapping is used in the simulation:

- Traffic Lights (PORTA):
  - NS: Red = PA0, Yellow = PA1, Green = PA2

- EW: Red = PA3, Yellow = PA4, Green = PA5

- Push Buttons (external interrupts, falling edge):
  - INT0 = PD2, INT1 = PD3, INT2 = PB2
  - Internal pull-ups are enabled; each button is connected to GND.

- Traffic Sensor:
  - PC1..PC0: 00 (normal), 01 (NS congested), 10 (EW congested)

0.0.2 Proteus Circuit Notes

- Crystal connected between XTAL1 and XTAL2, with two 22pF capacitors to GND.
- Each LED is connected as: `PAx` → 330Ω → LED → GND.

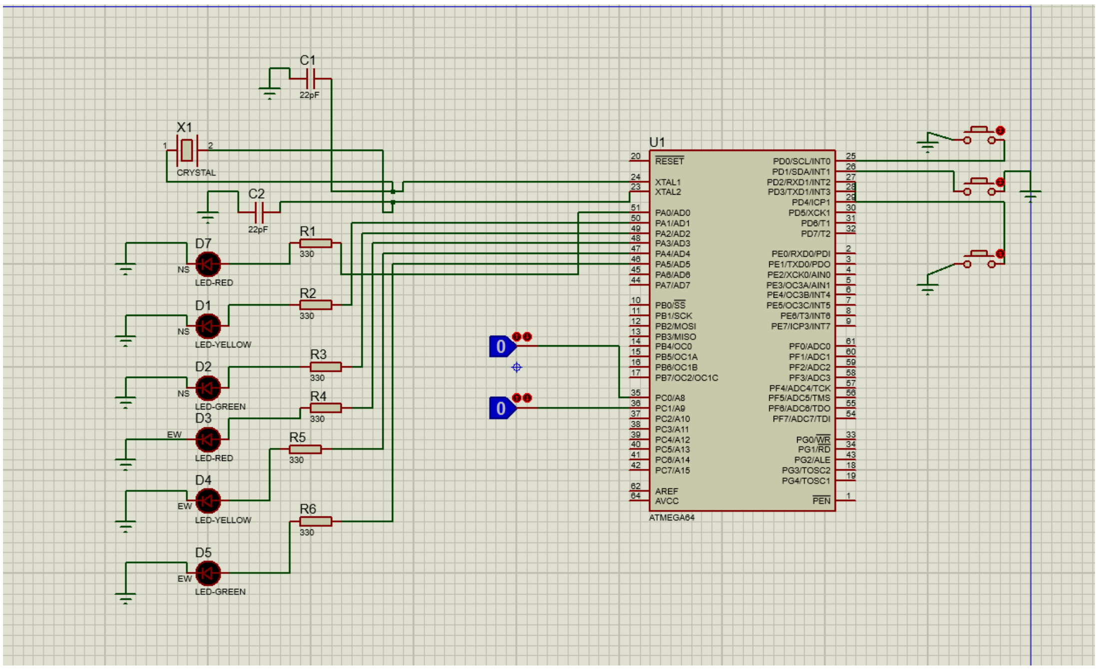


Figure 1: Proteus circuit topology (ATmega64 + LEDs + Buttons + Sensor).

Program Logic Description

The controller is designed as a **four-state FSM** that runs cyclically:

$$\text{NS\_Green} \rightarrow \text{NS\_Yellow} \rightarrow \text{EW\_Green} \rightarrow \text{EW\_Yellow} \rightarrow \text{NS\_Green} \rightarrow \cdots$$

To achieve accurate timing without blocking delays, **Timer1 is configured in CTC mode** to generate an interrupt every **10 ms**. A phase counter (loaded at the start of each state) is decremented on every timer tick. When the counter reaches zero, a “phase-done” flag is set. The main loop monitors this flag, updates the FSM state, applies the corresponding traffic light outputs, and reloads the next phase duration.

This structure ensures deterministic timing and stable operation while allowing safe real-time updates of timing values.

Timing Design

0.0.3 Timer1 (CTC) Tick

Timer1 is configured to generate a 10 ms tick. The design uses:

- A fixed interrupt period (10 ms).
- A 16-bit phase counter: duration in ticks = seconds $\times 100$.

0.0.4 State Durations

- Green durations are stored in seconds: G_{NS} and G_{EW}.
- Yellow durations are stored in ticks (10 ms units): Y_{NS} and Y_{EW}.

Time Adjustment Using External Interrupts

Three push buttons are connected to `INT0`, `INT1`, and `INT2` (falling edge). When an interrupt occurs, only the start of the press is recorded. The press duration is measured using the periodic 10 ms timer tick to avoid heavy processing inside external ISRs.

When the button is released, a *pending action* is generated and later executed in the main loop:

- `INT0` and `INT1`: threshold of $1.5\,s$ for short/long press. The action relatively increases one direction’s green time while decreasing the other direction’s green time.
- `INT2`: threshold of $1.0\,s$. The action adjusts yellow durations in steps of $250\,ms$ (25 ticks).

0.0.5 Timing Constraints

The following constraints are enforced:

- Green: 5 s $\leq G \leq$ 30 s
- Yellow: 2 s $\leq Y \leq$ 3 s

When the limit is reached, further presses do not modify the timing values.

Traffic Sensor Logic

Traffic density is read from pins `PC1..PC0`:

- `00`: normal mode
- `01`: heavy traffic in NS direction
- `10`: heavy traffic in EW direction

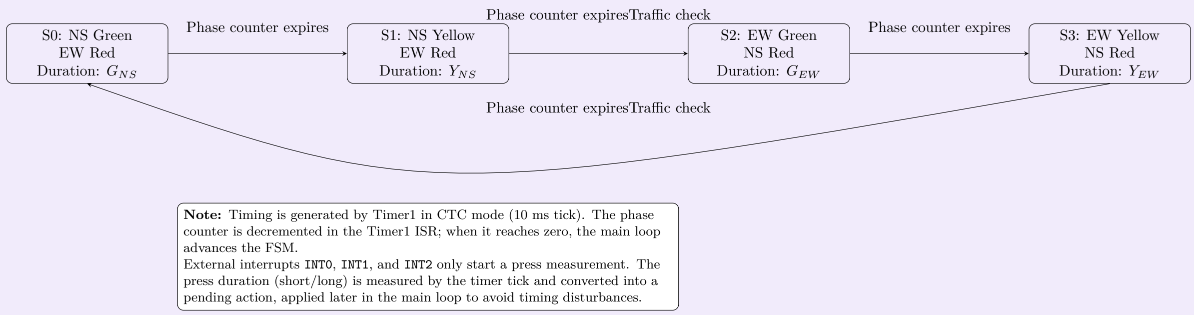
At safe transition points (before entering a green state), the program checks the sensor. If traffic is detected, the current timing parameters are saved and the controller enters a priority mode:

- Congested direction: Green → maximum, Red → minimum
- Opposite direction: Green → minimum, Red → maximum
- Yellow durations are forced to their minimum values

When the sensor returns to `00`, the previously saved timing values are restored.

Finite State Machine Diagram

Explanation:



Testing Procedure in Proteus

The following tests were performed:

- FSM correctness: sequence S0 → S1 → S2 → S3 → S0 repeats continuously.
- Timing accuracy: phase durations match the configured parameters based on the 10 ms timer tick.
- Button tests:

- INT0/INT1 short press vs. long press produces opposite timing changes.
- INT2 changes yellow times in 250 ms steps with clamp behavior.

- Traffic sensor tests:
  - PC1..PC0 = 01 forces NS priority (green max, opposite green min).
  - PC1..PC0 = 10 forces EW priority.
  - Returning to 00 restores the previously saved parameters.

Results and Discussion

The system operated continuously with stable timing. The interrupt-driven design prevented timing disturbances during button operations. Constraints were respected, and traffic sensor mode correctly saved/restored timing values.

Conclusion

A robust traffic light controller was implemented using AVR Assembly on ATmega64. The design is based on a deterministic FSM with precise timing from Timer1 CTC interrupts. External interrupts enable safe real-time adjustments, and a traffic sensor mode provides adaptive prioritization. The project was validated through Proteus simulation and meets the functional requirements.